WHITEPAPER · v1

Ambient

The workspace for your data.

Ambient is a workspace where pages, blocks, and typed collections compose into living documents — populated by pluggable connections that import data from the systems you already use. v1 ships an opinionated trading journal on top of a generic platform that scales to any structured-data vertical without rewrites.

Document	Product & technical summary
Audience	Prospective investors and team members
Status	Confidential — for review only
Date	May 2026
Version	v1

Contents

1. Executive summary

Ambient is a multi-tenant SaaS workspace for structured data. It combines three primitives — pages, blocks, and collections — with a fourth concept, connections, that import data from external systems and feed the same collections users create by hand. Together they let a user assemble a document, a dashboard, and a database in the same workspace.

Our v1 product ships an opinionated trading journal as the first vertical: it ingests Interactive Brokers Activity Statement CSVs, aggregates raw fills into round-trip trades, and surfaces them through pre-built dashboards. Every part of this experience is built on generic platform primitives — the only trading-specific code is a CSV parser, an aggregation pipeline, and three template files. Adding a new vertical is a matter of adding a Connection and a Template, not rebuilding the platform.

The product is built on Next.js 15, Supabase (Auth + Postgres + Storage), and the Plate.js block editor. Security is enforced by Postgres Row-Level Security, not by application code. Server Components and Server Actions are the only ways to read and write user data — there are no REST APIs to misuse.

> Why Ambient matters: Notion is a generic workspace with no data model. Airtable is a database with a weak editor. Vertical SaaS tools are siloed by industry. Ambient is the first workspace that treats importable, typed data as a first-class primitive — turning a single product into a platform for any vertical that lives in spreadsheets today.

2. The problem

Knowledge workers live inside two unrelated tools. The first is a document editor — Notion, Confluence, Google Docs — where they write, think, and share. The second is a spreadsheet or database — Excel, Airtable, a vertical SaaS — where their structured data lives. Connecting the two is manual, brittle, and constantly out of date.

The result is everyday friction: KPIs in slide decks that no one remembers to refresh, dashboards bolted onto data warehouses that take an engineer to change, journals and review documents that quietly fall out of sync with the underlying numbers.

Existing tools each miss one half

Tool	Strength	What it lacks
Notion / Confluence	Beautiful block editor; calm document UX	No real data model; no importers; no typed aggregations

Tool	Strength	What it lacks
Airtable / Coda	Rich typed databases; formula columns	Editor is an afterthought; pricing scales poorly per workspace
Vertical SaaS (e.g., TraderSync, Edgewonk)	Domain templates, importers, KPIs out of the box	Locked to one vertical; you cannot bring your own data shape
BI tools (Looker, Metabase)	Powerful analytics on warehouses	Not a workspace; not editable by the person doing the work

Each tool optimizes for one half of the problem. The user pays for all of them and stitches the gap by hand.

3. Ambient in one paragraph

> Ambient is a workspace where you can write a document, define a typed collection, and import data from external systems — all on the same canvas, secured by Row-Level Security, and built so that adding a new vertical is additive, not invasive.

4. Product overview

Ambient is organized around four primitives. Each one is generic; trading-specific behavior is layered on top.

Pages

A page is the top-level object in the sidebar. It has one of two types:

- Dashboard — a Plate.js block document containing text, images, and data-aware blocks (Card, Chart, Table, Row).
- Collection — a typed list view backed by a schema (text, number, currency, date, datetime, select, multi-select, checkbox).

Blocks

Blocks live inside dashboard pages. The four data-aware blocks all read from a collection, referenced by ID:

- Card — one metric (count, sum, average, percentage) over a filtered collection.

- Chart — line, bar, or area chart of grouped aggregations.
- Table — a slice of a collection's list view, with sort, filter, and visible-field configuration.
- Row — a single collection row rendered inline, with all of its fields.

No block type is trading-specific. The Trade detail UI is the generic Row block pointed at a row in the system-managed Trades collection.

Collections

Collections are universal. Users can create them by hand or have them produced by a connection. The data lives in a generic JSONB row store with a typed schema. The same primitive backs a user's homemade 'Books I'm reading' list and the IBKR connection's system-managed Trades table.

Connections

A connection is a pluggable importer. It declares the file types it parses, the collections it produces, optional per-connection settings, and an optional post-process pipeline that runs after import (e.g., aggregating raw fills into round-trip trades). v1 ships one connection — Interactive Brokers Activity Statement. Adding more is purely additive.

These four primitives compose to form every product surface in v1:

- A blank workspace with a Cmd+K palette to create pages.
- Apply a template (saved page recipe) to instantiate a pre-built dashboard.
- Import data from a connection to populate system collections.
- Mix system data with your own collections, your own fields, your own notes.

5. System architecture

Ambient is a server-rendered Next.js application backed by Supabase. The architecture is deliberately boring — Server Components do reads, Server Actions do writes, and Postgres Row-Level Security is the security boundary.

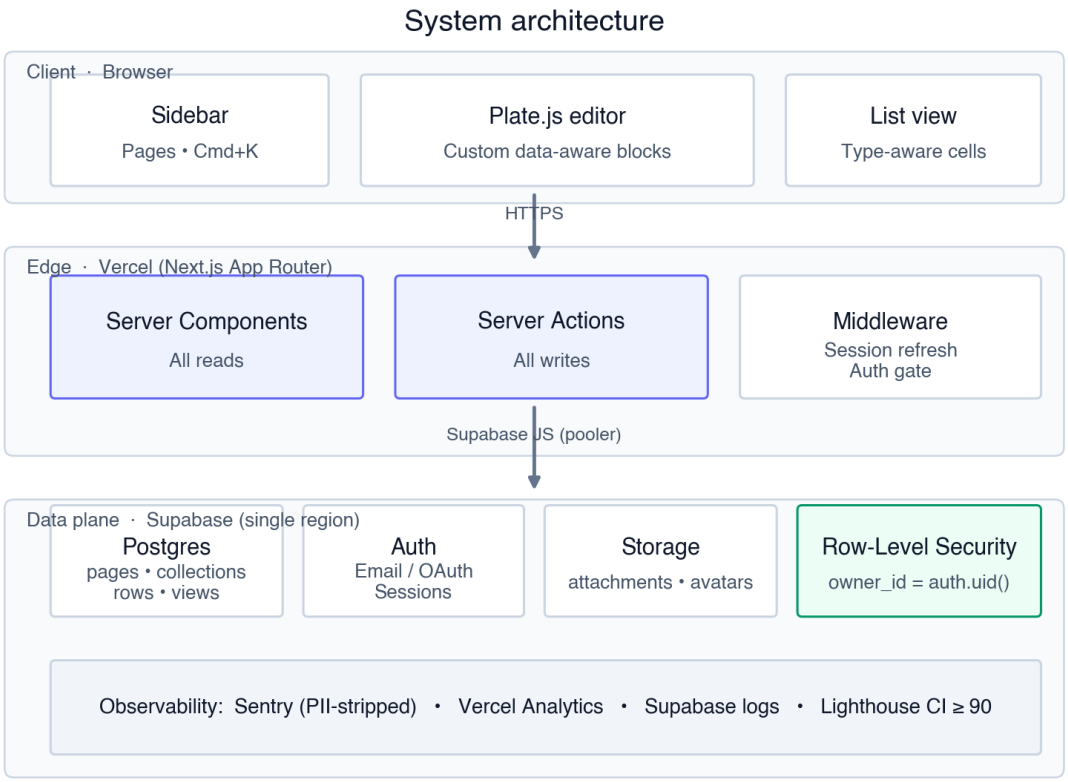

Figure 1. Three-tier architecture. Server Components own reads; Server Actions own writes; RLS is enforced inside Postgres on every owned table.

Key architectural choices

Server Components for reads	Data fetching colocated with rendering; no client-side state library; React's RSC cache deduplicates fetches across nested blocks.
Server Actions for writes	Type-safe, RPC-style writes. No REST API surface to misuse or to keep in sync with the client.
RLS as the boundary	Every owned table has a policy: `owner_type = 'user' AND owner_id = auth.uid()`. Forgetting a WHERE clause cannot leak data.

Soft delete by default	Pages and collections have `deleted_at`; standard queries filter it out. Trash & Restore is a v2 UI feature with no migration needed.
One region, single database	Same-region Vercel + Supabase. Multi-region is an enterprise-tier concern; we are explicit about the latency trade-off.

6. Data model

Ambient stores all structured user data in a small, generic schema. The same tables back user-created collections, system collections produced by connections, and any future view types (kanban, calendar, gallery) that we ship.

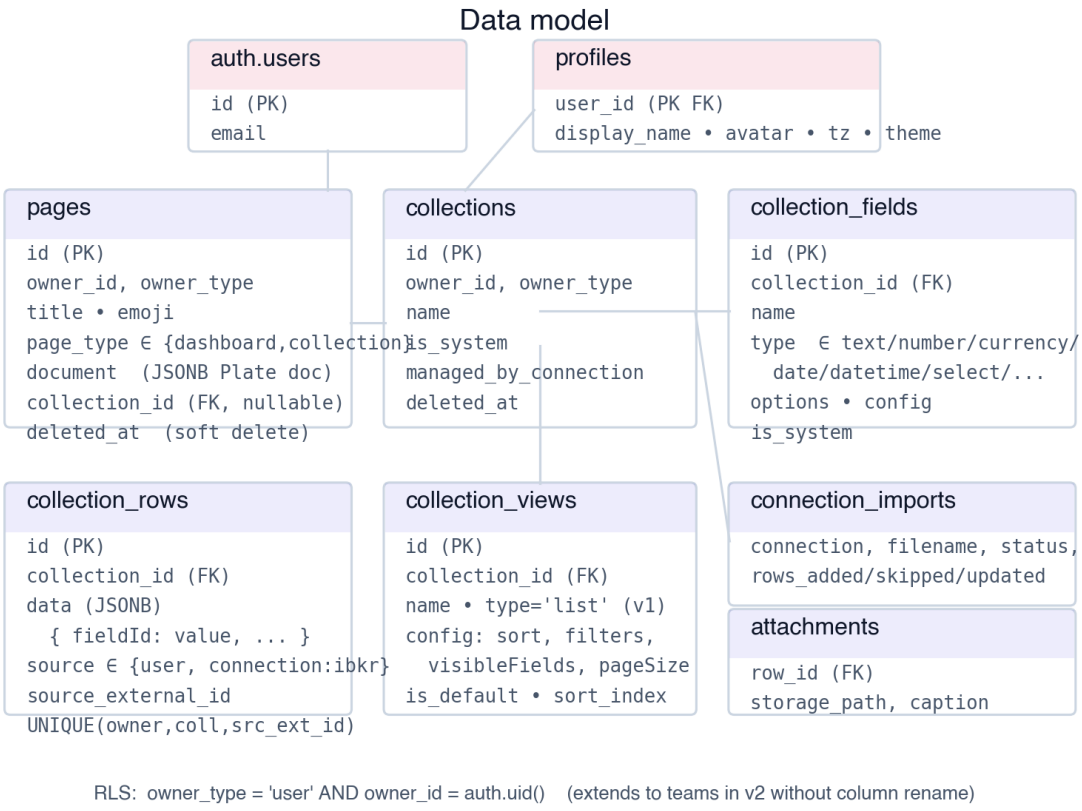

Figure 2. Core schema. All owned tables carry `owner_id` + `owner_type`; v1 uses `'user'`, v2 will extend the same policy to workspaces with zero migrations.

Forward-compatible ownership

Every content table uses an owner_id + owner_type pair instead of a bare user_id. v1 always sets owner_type = 'user'. When Ambient ships teams in v2, the same column accepts owner_type = 'workspace' and the RLS policy extends with an OR clause — no column rename, no migration of existing

rows.

Typed fields

Field type	Stored shape	Notes
text	string	—
number	number	Configurable precision and 'decimal'/'percent' format.
currency	{ amount, currency_code }	Multi-currency-aware. Aggregations group by code in v1; FX in v2.
date	ISO date	—
datetime	ISO timestamp (UTC)	Always stored in UTC; rendered in the user's profile timezone.
select / multi_select	string / string[]	Option list with name, value, and color stored per field.
checkbox	boolean	—

All row data is stored as a single JSONB column keyed by field id. Expression indexes are added per-collection on hot fields when needed. At v1 volumes (thousands of rows per user) we have measured sub-50ms aggregations against this layout — no materialized views are required to hit our latency targets.

7. The platform seam

The single most important architectural decision in Ambient is the boundary between blocks and data. Every data-aware block — Card, Chart, Table, Row — talks to a typed Collection interface, never to a specific table or domain concept:

```
interface Collection {
  list(opts): Promise<Row[]>
  count(opts): Promise<number>
  aggregate(opts): Promise<AggregateResult>
  getRow(rowId): Promise<Row | null>
}
```

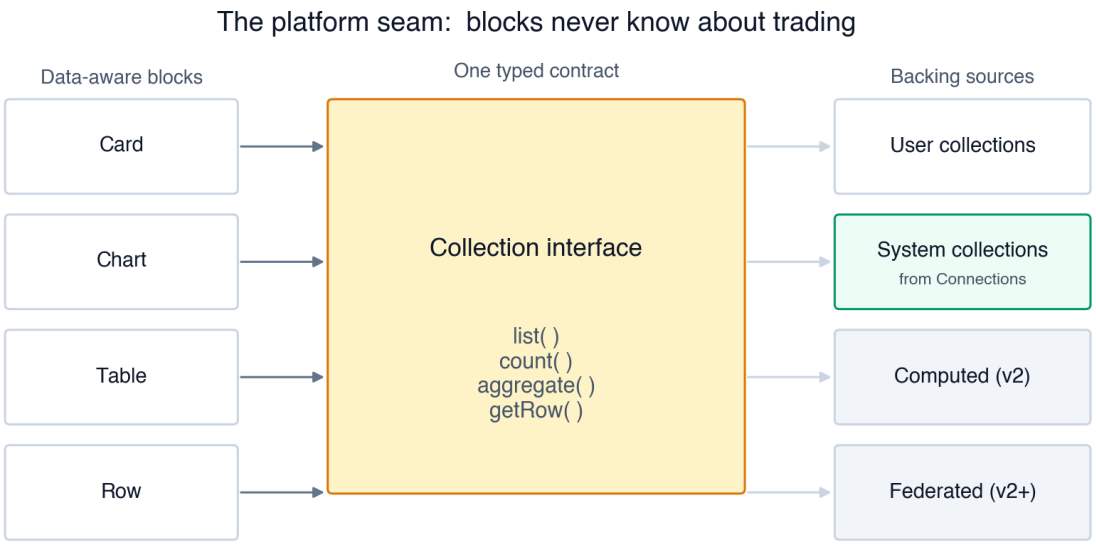

Figure 3. Blocks bind to the Collection interface, not to underlying tables. New backing sources (computed collections, federated sources) are additive.

Implication for investors: the marginal cost of a new vertical is a Connection (parser + pipeline) and a Template (page recipe). Everything the user sees — pages, blocks, the editor, the list view, security, settings — is shared platform code.

8. The first vertical — Trading

v1 ships an opinionated trading journal — the first user-facing surface of the platform. We picked trading deliberately:

- High-signal data. Every fill has a timestamp, price, and quantity. There is no ambiguity, so the aggregation pipeline can be tested deterministically.
- Painful status quo. Existing trading journals are siloed and ugly; users routinely export CSVs and rebuild dashboards in Excel.
- Repeat usage. Active traders review performance daily, weekly, and monthly — a workflow that benefits from a workspace, not a one-off report.

Connection: IBKR Activity Statement

The connection parses Interactive Brokers' multi-section Activity Statement CSV. It is section-aware, timezone-aware (configurable per connection; default America/New_York), and idempotent — re-importing the same file adds zero new rows. Non-stock asset categories are silently skipped and recorded in an audit table.

Pipeline: Fills → Trades

After fills are upserted, an aggregator walks fills per symbol in chronological order and groups them into round-trip trades (flat-to-flat). The algorithm handles scaling in, partial closes, exact closes, and direction flips — a single fill that crosses zero is split into a closing portion and an opening portion for the new direction. Per-trade P&L is computed using weighted-average entry and exit prices.

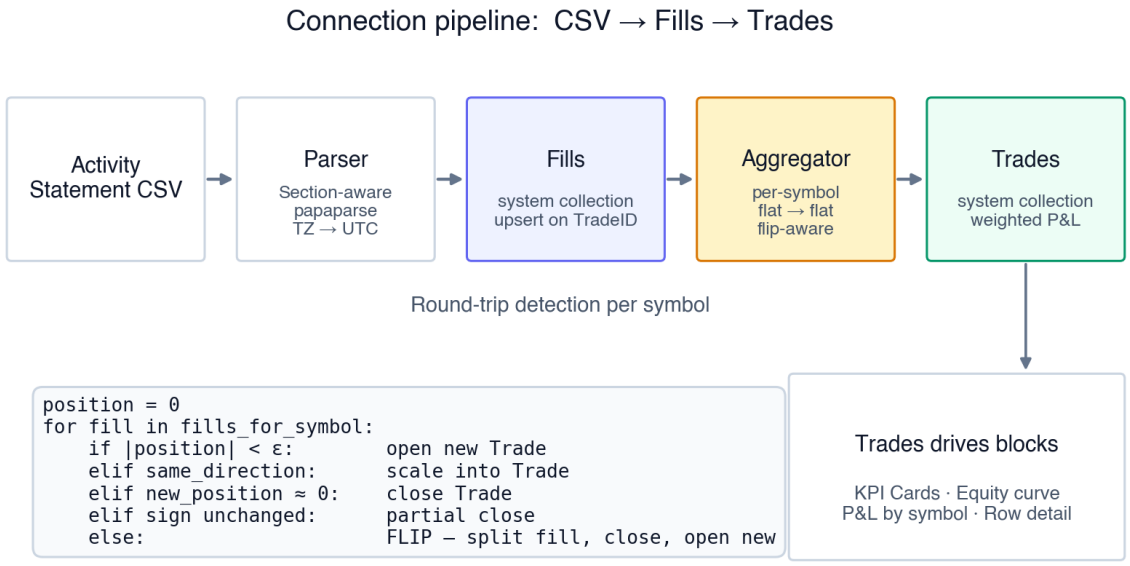

Figure 4. The IBKR pipeline. CSV → parsed Fills → round-trip Trades — each step is a pure function, fully unit-tested, and idempotent.

Templates: pre-built dashboards

Three trading templates ship with v1 — Performance Dashboard, Daily Journal, and Weekly Review. Each is stored as a JSON file in the repo and applied via + New page → Apply template. Templates compose the same generic blocks that any user can drop into any page.

9. Security & multi-tenancy

Ambient is multi-tenant from day one. We treat data isolation as a security boundary, not a feature.

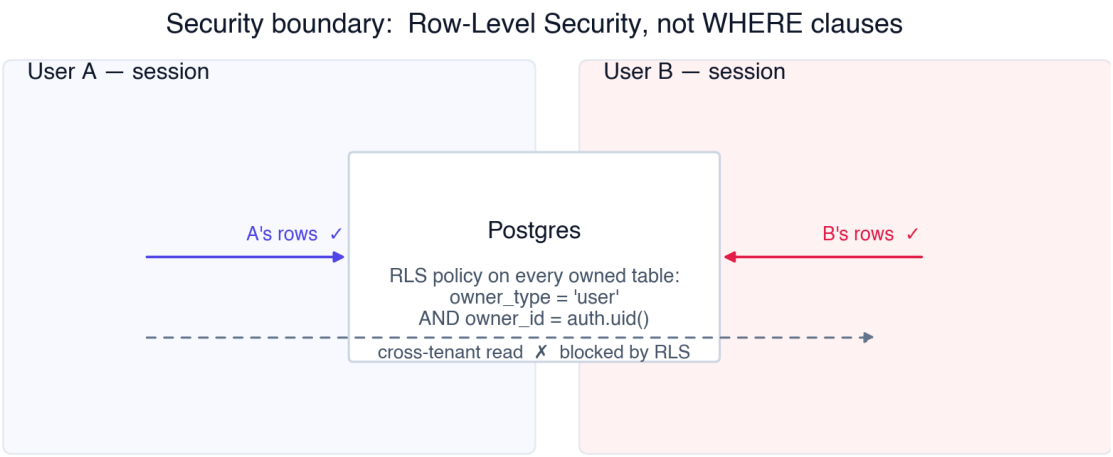

Figure 5. Every read and write is filtered by Postgres RLS on the user's session identity. Cross-tenant reads are blocked at the database, not by application code.

Three Supabase clients

Client	Where	Permissions
Browser client	Client Components	RLS-bound to `auth.uid()`
Server client	Server Components, Server Actions	RLS-bound to the user's session
Admin client	Service-role-only Server Actions (account deletion, seed reset)	Bypasses RLS; gated by folder convention + lint rule

Storage

Two private Supabase Storage buckets — attachments and avatars — both with RLS policies that limit reads and writes to paths prefixed by the user's id.

Auth

Email/password and Google OAuth via Supabase Auth. Email-change requires confirmation; delete-account triggers an admin Server Action that cascades through foreign keys. 2FA is supported by Supabase and ships in v2.

10. Performance

Latency	Same-region Vercel + Supabase deployments. Server Components stream first paint.
Connections	All server-side connections use the Supabase pooler URL — no per-request connection storms under serverless load.
Bundle size	Per-route code splitting. The Plate.js editor and Recharts are dynamic-imported and never reach the marketing or auth bundles.
Re-renders	Plate's onChange is debounced 500ms before persisting; the React RSC cache deduplicates identical fetches across blocks on the same page.
Save safety	Optimistic concurrency on page documents; pending debounced saves are force-flushed on navigate and beforeunload.
Quality gate	Lighthouse CI runs on every PR — Performance score below 90 on the dashboard route fails the build.

11. Technology stack

Layer	Choice	Why
Framework	Next.js 15 (App Router)	Server Components fit the latency bar; mature SaaS ecosystem.
UI library	shadcn/ui + Tailwind	Owned, copy-paste primitives; accessible by default; theme-able.
Editor	Plate.js (Slate-based)	First-class shadcn integration; granular plugins for our data-aware blocks.
Charts	Recharts via shadcn Chart wrapper	Lazy-loaded only where used.
Backend	Supabase (Auth + Postgres + Storage)	One provider; RLS-aware; predictable cost curve.
State	None (server-owned)	Server Components + URL state; no client store library to maintain.
Testing	Vitest + Playwright + axe-core	Unit, integration (RLS), and end-to-end coverage on every PR.
Observability	Sentry, Vercel Analytics, Supabase logs, Lighthouse CI	Minimal but complete — enough to debug a customer incident.

12. Roadmap

Ambient's roadmap is organized around verticals, not features. Every release either deepens an existing vertical or adds a new one — the platform itself grows by necessity, not by speculation.

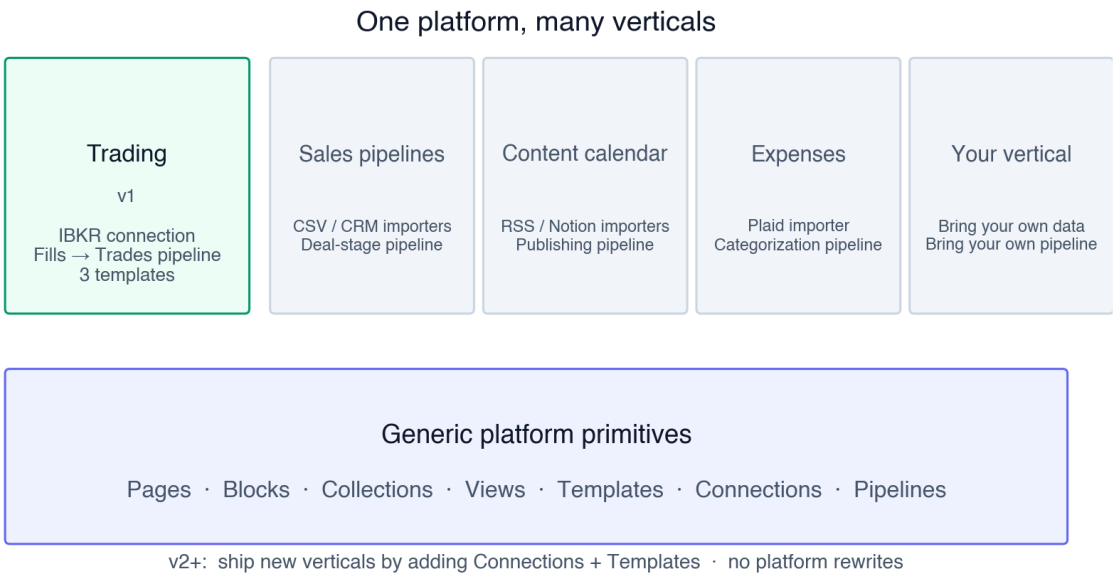

Figure 6. Trading is the first vertical; future verticals plug in at the Connection + Template layer without changing the platform core.

Phase	Focus	Highlights
v1 (now)	Trading vertical, core platform	IBKR connection, Fills → Trades, 3 templates, Cmd+K, settings, RLS.
v1.5	Trading depth, polish	Additional IBKR sections (cash, fees, dividends), more templates, mobile read.
v2	Teams, billing, marketing	`owner_type='workspace'`, Stripe, public landing + docs.
v2.5	Second vertical (TBD)	Sales pipeline or content calendar — chosen from inbound signal.
v3+	Platform features	Computed collections, inter-collection relations, view types (kanban, calendar).

13. Why now

Three shifts have made Ambient possible — and necessary — at the same time:

• Server-rendered React is finally fast. Next.js Server Components close the gap with classic server-rendered apps while keeping the React component model. The Notion-class editor experience no longer requires a Notion-class engineering team.

• RLS-first databases are production-ready. Supabase has matured Row-Level Security from a power-user feature to a reliable security boundary. Multi-tenancy is now a default, not a milestone.

• Block editors are commoditized. Plate.js and similar libraries make it possible to ship a serious block editor as a single dependency. The differentiator is no longer the editor itself — it is what the editor reads from.

Ambient bets that the next workspace is not a better Notion or a prettier Airtable, but a workspace where importable, typed data is the primitive — and where every new vertical is a Connection, not a fork.

Appendix A — Design decisions

Decision	Choice	Rationale
Storage shape for rows	JSONB keyed by field id	One primitive for user collections and system collections; expression indexes per-field as needed.
Ownership column	`owner_id` + `owner_type`	Forward-compatible to v2 teams without a column rename.
P&L math	Weighted average	Correct for performance review. FIFO matters for taxes; out of v1 scope; isolated for swap.
Round-trip rule	Flat → flat per symbol	Matches the user's mental model; flip handling makes it deterministic on every CSV.
Editor	Plate.js	Slate-based, granular plugins, shadcn integration; we own the data-aware blocks layer.
Realtime / collab	Deferred to v2	Single-user editing covers v1 use cases; CRDT layer is invasive enough to be its own milestone.
Background queue	Deferred — synchronous Server Actions in v1	v1 imports are bounded (≤10k fills); queue is a v2 concern when we add larger feeds.
Materialized views	None in v1	Direct queries with indexes hit our latency bar; add only if metrics demand it.

Appendix B — Glossary

Page	A top-level sidebar item — either a Dashboard or a Collection.
Dashboard	A page containing a Plate.js block document.
Collection	A typed table (schema + rows) shown via a list view.
Block	An element in a Plate document. Data-aware blocks: Card, Chart, Table, Row.
View	A saved configuration (sort, filters, visible fields) for a collection.
Connection	A pluggable importer that produces collections from external data.
Template	A saved page recipe applied via + New page → Apply template.
Pipeline	A named transform run after a connection imports — e.g., Fills → Trades.

Round-trip	One trade, defined as flat → position → flat on a single symbol.
System collection	Created and managed by a connection; users can add adjacent fields but cannot rename or delete the system fields.